

Author: Maria Loisa Reyes

Course: AIM PostGrad Artificial Intelligence

Date: 28/9/2026

Live app: <https://graded-mini-project-14-3-next-js.vercel.app>

Repository: <https://github.com/MariaReyes99/Graded-Mini-Project-14.3-NextJS>

Reflection: Acme Widget API RAG Assistant

Week 14.3 Graded Mini Project

For this project I built the Acme Widget API Field Guide, a public RAG assistant that answers developer questions about the Acme Widget API and the internet standards it is built on, citing the source behind every answer. I started from the Week 14 Next.js RAG starter. The corpus has two parts. The first is eleven Markdown documents derived from the synthetic Acme Widget API Specification v4.2 supplied with the starter, including two internal emails (onboarding and an API key deprecation notice). I chose the Acme specification because I know its ground truth, so I can verify every answer. However, my first version was only about 3,000 characters, and even after expanding it with the full specification it was small and entirely synthetic. To make the corpus meaningful, I added the seven real public standards the API relies on: RFC 6749 (OAuth 2.0), RFC 6750 (Bearer tokens), RFC 7519 (JWT), RFC 6585 (HTTP 429), RFC 8594 (Sunset header), RFC 3339 (timestamps) and the Idempotency-Key Internet-Draft. The IETF licence allows these to be copied in full but not modified, so the files are stored exactly as published and all cleanup happens in code. The result is 18 documents and 380 chunks, and a bot that explains both how Acme behaves and the standards behind it.

Embeddings use text-embedding-3-small stored in Upstash Vector, and answers use gpt-4o-mini, chosen for low cost and fast streaming on a public app, since answers depend on retrieved facts rather than deep reasoning. My original plan was fixed 500–700-token chunks, but the two document types are structured differently, so I chunk each to match. Acme Markdown files are split on headings (58 chunks, about 350 characters each), with a 3,000-character limit kept only as a safety net. RFCs are long plain-text documents with numbered sections, so each section, such as “4.4 Client Credentials Grant”, becomes a chunk, and sections over 1,500 characters are split with 200 characters of overlap (322 chunks, about 840 characters each). Page headers, footers and boilerplate such as copyright notices, tables of contents and reference lists are removed before embedding, and very short sections are merged into the next. Each chunk stores its file name, document title, section and kind (Acme or standard), and the title and section are prepended to the embedded text, which fixed an early problem where authentication questions retrieved endpoint content. A dry-run mode prints chunk statistics before any API calls, which let me check the RFC parsing cheaply.

Retrieval is a tool call, `getInformation`, which takes a query and a scope (acme, standards or both) that becomes a metadata filter. This was the key trade-off of adding the RFCs: they make up about 85% of the chunks and could otherwise crowd out the Acme documents. In testing, “How do I safely retry a POST /v4/orders request?” searched only the Acme docs and cited `Endpoints.md` and `Retries_and_Timeouts.md`; the OAuth 2.0 client credentials question searched only the standards and cited RFC 6749 §4.4; and “Is Idempotency-Key an official standard?” correctly said it is a work-in-progress draft. Greetings are answered without searching, and in earlier testing an out-of-scope GraphQL question was answered with “the documentation does not cover this.” Each Sources panel is labelled with the scope searched and shows the file, section, score and text, and I added Markdown rendering so answers show real lists and bold text instead of raw symbols. Deployment taught me that working locally is not working publicly: my first shared link was a deployment URL protected by Vercel Authentication, which incognito testing exposed, so I switched to the public production domain. Because the endpoint is public, I added request validation, a 2,000-character question limit and a history cap. My stretch goal was suggested-prompt chips, which help first-time users start with a well-formed question.

The project has clear limitations. The Acme specification is synthetic, so the bot has never faced real developer questions. Retrieval always returns five chunks even when matches are weak, and choosing the right scope depends on the model following its instructions. The model sometimes cites a document's title instead of its file name, and the Idempotency-Key source is a draft that may change. All testing was manual. Next, I would build an evaluation set of question-and-answer pairs to measure retrieval accuracy per scope, compare chunk sizes and hybrid keyword-plus-vector retrieval, add a minimum relevance score, and add per-user rate limiting. Overall, the project taught me that corpus design and chunking decisions matter as much as the choice of model.